

The Complete Syllabus

Data Science · Generative AI · Transformers — Zero to Industry-Ready

This is the master reference syllabus — the single document to study and track against. It is sequenced so each tier builds on the last, and it covers the full path from absolute beginner to an industry-ready, application-focused AI engineer. Work top to bottom, and build a project at the end of every tier. Application beats re-reading.

How to read this document

- Tiers run in order; each assumes the previous one is solid.
- ★ marks the highest-priority topics for a transformers / GenAI goal.
- ♦ Mandatory ■ Optional / situational for an application-focused engineer.
- Tiers 0–5 are the core path. Tier 6 is an optional/just-in-time appendix.

Tier	Focus	Time
0	Programming & Tooling Foundations	3–4 wks
1	Mathematics for Data Science	10–12 wks
2	Core Machine Learning	4–5 wks
3	Deep Learning	4–5 wks
4	NLP → Transformers → Generative AI	8–10 wks
5	Engineering, Evaluation & Production	3–4 wks
6	Optional / Situational & Research Depth	ongoing

Tier 0 · Programming & Tooling Foundations

3–4 wks

Phase 0.1 — Python Core 1–2 wks

- Syntax, variables, data types (int, float, str, bool, None)
- Operators: arithmetic, comparison, logical
- Strings: indexing, slicing, methods, f-strings
- Data structures: list, tuple, dict, set — and when to use each
- Control flow: if/elif/else, for, while, break/continue
- Functions: arguments, defaults, return, scope, *args/**kwargs
- Comprehensions (list/dict/set), generators (yield), decorators
- Classes & OOP: `__init__`, self, attributes, methods, inheritance
- Error handling: try/except/finally, raising exceptions
- Context managers (with), file I/O
- Modules, imports, project structure
- Virtual environments (venv/conda), pip, requirements.txt
- Debugging basics and reading tracebacks
- Type hints and basic testing (assert / pytest intro)

■ **Goal:** *Read and write clean, idiomatic Python without looking up basics.* ♦ **Mandatory**

Phase 0.2 — Scientific Python Stack 1–2 wks

- NumPy: arrays, broadcasting, vectorized operations, indexing
- Pandas: DataFrames, indexing, groupby, merging/joining, cleaning messy data
- Matplotlib & Seaborn: distributions, relationships, dashboards
- scikit-learn: the core ML library — estimators, fit/predict API, pipelines
- Data preprocessing utilities: train_test_split, StandardScaler, encoders
- Jupyter notebooks; Git/GitHub for version control
- Loading data from CSV/JSON/Excel; basic SQL querying

■ **Goal:** *Load a raw CSV, clean it, explore it, plot insights, and run a first model end-to-end.* ♦ **Mandatory**

Tier 1 · Mathematics for Data Science

10–12 wks

Weight effort toward **Linear Algebra** and **Calculus** for a transformers/GenAI goal. Intuition first, but rigorous once introduced.

Phase 1.1 — Foundations (refresher) 1–2 wks

- Functions, exponents, logarithms (log-loss, log-likelihood, log scales)
- Summation & product notation (Σ , Π)
- Sets and basic combinatorics (permutations, combinations, factorials)
- Inequalities, absolute value, and interval notation
- Sequences and series basics

Phase 1.2 — Linear Algebra ★ most important 3 wks

- Vectors: addition, scaling, dot product, norms (L_1 , L_2 , L_∞)
- Angle between vectors, cosine similarity, orthogonality
- Linear combinations, span, linear independence
- Vector spaces, basis, and dimension
- Matrices: addition, multiplication, transpose, identity, inverse
- Special matrices: diagonal, symmetric, orthogonal
- Determinants and their meaning (area/volume scaling, invertibility)
- Matrix rank and what it tells you
- Solving systems of linear equations (Gaussian elimination)
- Matrix–vector products as linear transformations
- Orthogonality, projections, and orthonormal bases

- Gram–Schmidt process (orthonormalization)
- Eigenvalues & eigenvectors; the characteristic equation
- Diagonalization
- Decompositions: SVD (singular value decomposition) and PCA
- QR decomposition (overview)

■ **Goal:** *Understand why embeddings, attention, and every neural layer are matrix operations.* ♦ **Mandatory** ★

Phase 1.3 — Calculus 2–3 wks

- Limits and continuity (conceptual)
- Derivatives and rules (power, product, quotient, chain)
- Partial derivatives and the gradient (∇)
- The chain rule, deeply — this IS backpropagation
- Jacobian and Hessian matrices (what they are, why they matter)
- The gradient as direction of steepest ascent — geometric meaning
- Maxima/minima, critical points, optimization basics
- Convexity (intro) and saddle points
- Taylor series approximation (first/second order intuition)
- Gradient descent intuition

■ **Goal:** *Be fluent with gradients and the chain rule — the machinery of model training.* ♦ **Mandatory** ★

Phase 1.4 — Probability 3 wks

- Sample spaces, events, axioms of probability
- Conditional probability, independence, Bayes' theorem
- Random variables: discrete vs continuous
- PMF, PDF, and CDF
- Key distributions: Bernoulli, binomial, normal, Poisson, uniform, exponential
- When each distribution applies (modeling intuition)
- Expectation, variance, standard deviation
- Covariance and correlation
- Joint, marginal, and conditional distributions
- Law of large numbers
- Maximum likelihood (MLE) vs maximum a posteriori (MAP) — the link to Bayes

■ **Goal:** *Reason about uncertainty and probability — models output probabilities.* ♦ **Mandatory**

Phase 1.5 — Statistics 3–4 wks

- Descriptive stats: mean, median, mode, variance, std, quartiles, IQR
- Sampling, sampling distributions, the Central Limit Theorem
- Estimation: point estimates, standard error, confidence intervals
- Hypothesis testing: null/alternative, z-tests, t-tests, p-values, critical values
- Type I / Type II errors and statistical power
- ANOVA, chi-square tests
- Maximum Likelihood Estimation (MLE)
- Regression from a statistical view (assumptions, residuals)
- Correlation vs causation; confounders
- Bootstrap and resampling methods
- Bias, consistency, and the bias–variance decomposition (statistical view)

■ **Goal:** *Draw honest conclusions from data and underpin model evaluation.* ♦ **Mandatory** (some parts ■)

Phase 1.6 — Optimization & Information Theory 2 wks

- Convex vs non-convex functions; local vs global minima
- Gradient descent variants: batch, stochastic, mini-batch
- Momentum, RMSProp, Adam
- Learning-rate schedules and warmup
- The vanishing/exploding gradient problem (intro)
- Loss/cost functions: MSE, MAE, cross-entropy
- Entropy, cross-entropy, KL divergence (core to classification & LLM losses)

- Regularization as optimization (L1/L2 penalties)
- **Goal:** *Understand how models minimize error — the engine under all training.* ♦ Mandatory ★

Tier 2 · Core Machine Learning

4–5 wks

Use scikit-learn end-to-end. Classical ML grounds everything that follows.

Phase 2.1 — ML Foundations 2 wks

- Supervised vs unsupervised vs reinforcement learning
- Classification vs regression
- The ML workflow: data → features → train → evaluate → deploy
- The bias–variance tradeoff
- Overfitting & underfitting; generalization
- Train / validation / test splits
- Cross-validation (k-fold)
- Regularization (L1/Lasso, L2/Ridge, elastic net)
- Data leakage — what it is and how to avoid it

Phase 2.2 — Feature Engineering & Data Prep 1 wk

- Feature scaling: standardization (z-score) & normalization (min-max)
- Encoding categoricals: one-hot, label, ordinal, target encoding
- Handling missing data: imputation strategies
- Outlier detection and treatment
- Feature creation, binning, and transformations (log, polynomial)
- Handling imbalanced data: resampling, SMOTE, class weights
- Feature selection methods (filter, wrapper, embedded)
- Building scikit-learn Pipelines to prevent leakage

■ **Goal:** *Turn raw, messy data into model-ready features — often more impactful than the algorithm.* ♦ Mandatory

Phase 2.3 — Core Algorithms 2–3 wks

- Linear & logistic regression
- Decision trees; random forests; gradient boosting (XGBoost / LightGBM)
- k-Nearest Neighbors (k-NN); Support Vector Machines (SVM); Naive Bayes
- Unsupervised: k-means, hierarchical clustering, DBSCAN
- PCA for dimensionality reduction
- Ensemble methods: bagging, boosting, stacking (overview)
- Evaluation: accuracy, precision/recall, F1, ROC-AUC, confusion matrix
- Regression metrics: MSE, RMSE, MAE, R²
- Hyperparameter tuning: grid search, random search, cross-validated search
- Model interpretability: feature importance, SHAP (intro)

■ **Goal:** *Frame a real problem, engineer features, tune, evaluate, and explain the result.* ♦ Mandatory

Tier 3 · Deep Learning

4–5 wks

PyTorch is the preferred framework for GenAI work.

Phase 3.1 — Neural Network Fundamentals 2 wks

- Neurons, layers, weights, biases
- Activation functions: ReLU, sigmoid, tanh, softmax
- Forward pass, loss, backpropagation, the training loop
- Optimizers (SGD, Adam), learning rates, batching, epochs
- Weight initialization (why it matters; Xavier/He)
- Vanishing/exploding gradients and how to mitigate them
- Regularization: dropout, batch normalization, early stopping
- Loss functions in practice; gradient checking (concept)

- PyTorch basics: tensors, autograd, nn.Module, training loop
- **Goal:** *Build and train a neural network from scratch in PyTorch.* ♦ **Mandatory** ★

Phase 3.2 — Architectures 2–3 wks

- CNNs for images: convolution, pooling, filters, transfer learning
- RNNs / LSTMs / GRUs for sequences — and why long-range dependencies are hard
- Encoder–decoder (sequence-to-sequence) architecture
- Embeddings as learned vector representations (the bridge to NLP)
- The attention mechanism (first intuition, before transformers)
- Build several models from scratch in PyTorch
- **Goal:** *Train a CNN and an RNN yourself, and understand the limits that motivated transformers.* ♦ **Mandatory**

Tier 4 · NLP → Transformers → Generative AI

8–10 wks

The core destination. Where everything converges.

Phase 4.1 — NLP Fundamentals 2–3 wks

- Text preprocessing: cleaning, stemming, lemmatization, stop-words
- Tokenization: word, subword (BPE, WordPiece), character
- Vocabularies and n-grams
- Bag-of-words and TF-IDF
- Word embeddings: Word2Vec, GloVe (meaning as vectors)
- Contextual vs static embeddings
- Sequence modeling tasks: classification, NER, translation, summarization

Phase 4.2 — Transformers ★ the core 3–4 wks

- The attention mechanism and self-attention — the single most important concept
- Queries, keys, values — how attention is computed
- Scaled dot-product attention (and why the scaling)
- Multi-head attention
- Positional encodings
- Layer normalization & residual connections
- The full transformer block (attention + feed-forward)
- Encoder vs decoder architectures (BERT vs GPT)
- Read 'The Illustrated Transformer' and the Attention Is All You Need paper
- Implement a tiny transformer (Karpathy's nanoGPT / 'Let's build GPT')
- **Goal:** *Explain, and code, how a transformer turns tokens into predictions.* ♦ **Mandatory** ★

Phase 4.3 — Large Language Models & Generative AI 3–4 wks

- Pretraining, model weights, scaling laws, context windows
- Tokenization at scale; temperature, top-k, top-p sampling
- How LLMs generate text (autoregressive decoding)
- Fine-tuning vs RAG; when to use which
- LoRA / PEFT; quantization
- Instruction tuning and RLHF (conceptual)
- Hugging Face ecosystem (transformers, datasets, tokenizers)
- Prompt engineering: zero/few-shot, chain-of-thought, system prompts
- Structured outputs and function/tool calling
- **Goal:** *Understand LLM behavior deeply enough to build reliably on top of it.* ♦ **Mandatory** ★

Phase 4.4 — Embeddings, Vector DBs & RAG 2–3 wks

- Embeddings in depth: generating, comparing (cosine similarity), storing
- Vector databases: Pinecone, Weaviate, Chroma, FAISS
- Indexing & approximate nearest neighbor (ANN) search
- RAG pipeline: ingest → chunk → embed → retrieve → generate

- Chunking strategies (size, overlap, semantic chunking)
- Retrieval techniques and re-ranking
- Context assembly and citation
- RAG evaluation: faithfulness, relevance, groundedness
- **Goal:** *Build a production-quality RAG system over a real corpus.* ♦ **Mandatory** ★

Phase 4.5 — AI Agents 2–3 wks

- Agents: tool use, the think–act–observe loop, multi-step reasoning
- Orchestration: LangChain / LangGraph
- Agent memory (short-term, long-term) and state
- Multi-agent patterns (overview)
- Prompt injection and agent security basics
- **Goal:** *Build a tool-using agent that solves multi-step tasks.* ♦ **Mandatory** ★

Tier 5 · Engineering, Evaluation & Production

3–4 wks

What turns a model-builder into an engineer who ships reliable systems. The most under-taught — and most hireable — part of the path.

Phase 5.1 — Software Engineering for ML

- Project structure and clean code for ML/AI projects
- Writing tests for data and models; reproducibility (seeds, fixed environments)
- Logging, configuration management, and secrets handling
- Experiment tracking (Weights & Biases / MLflow)
- Version control workflows; code review; CI basics
- **Goal:** *Work like an engineer, not just a notebook tinkerer.* ♦ **Mandatory**

Phase 5.2 — Data Engineering Basics

- Where data comes from: APIs, web scraping, databases
- SQL in depth (joins, aggregations, window functions)
- Data formats: JSON, CSV, Parquet
- Basic ETL / data pipelines
- Data validation and quality checks
- **Goal:** *Reliably source, move, and validate the data your apps depend on.* ♦ **Mandatory**

Phase 5.3 — Evaluation as a Discipline

- Building evaluation datasets and golden sets
- LLM-as-judge and automated evaluation
- Human evaluation and annotation
- A/B testing and online evaluation
- Regression testing for prompts and pipelines
- Tracking quality, hallucination, and faithfulness over time
- **Goal:** *Measure whether your GenAI system is actually good — and stays good.* ♦ **Mandatory** ★

Phase 5.4 — Deployment, MLOps & System Design

- Serving models behind an API (FastAPI)
- Containers (Docker) and basic cloud deployment
- System design for AI apps: scaling, concurrency, fallbacks, streaming
- Rate limits, cost management, and caching at scale
- Monitoring, observability, and alerting in production
- CI/CD for ML systems
- **Goal:** *Ship a deployed, monitored, cost-aware GenAI product.* ♦ **Mandatory**

Phase 5.5 — Responsible AI & Safety

- Bias and fairness in models and data
 - Privacy and data governance
 - Guardrails, content moderation, and safe deployment of LLMs
 - Transparency, documentation, and accountability
- **Goal:** *Deploy AI responsibly — increasingly expected in real roles.* ♦ *Mandatory*

Tier 6 · Optional / Situational & Research Depth

ongoing

Not required for the core application-engineer path. Learn any of these only when a specific project, role, or paper calls for it. ■ Optional.

Situational Applied Areas

- Time series analysis & forecasting (finance, demand, metrics)
- Recommender systems (collaborative & content-based filtering)
- Multimodal AI: vision-language models, image generation (diffusion), audio/speech
- Classical NLP depth: dependency parsing, POS tagging, topic modeling (LDA)
- Cloud platforms in depth: AWS / GCP / Azure and their AI services
- Graph machine learning (GNNs, knowledge graphs)

Advanced Mathematics (as needed)

- Multivariate calculus: Jacobians, Hessians, Lagrange multipliers
- Convex optimization theory: duality, KKT conditions
- Measure-theoretic probability (rigorous foundations — research only)
- Numerical methods: floating-point stability, conditioning
- Differential equations: ODEs/SDEs (diffusion & continuous-time models)

Advanced ML/DL & Research Competence

- Diffusion models & score-based generative models
- Mixture-of-Experts (MoE); advanced fine-tuning; distillation
- RLHF / DPO internals; reward modeling
- Mechanistic interpretability
- Distributed training, mixed precision
- Read papers fluently (arXiv, NeurIPS, ICML, ICLR, ACL); reproduce results; contribute to open source
- Write up and communicate original findings

■ **Goal:** *Move from consuming the field to contributing to it — the marker of research-level work.*

Project Spine — build one at the end of each tier

Tier	Project
0	CLI tool that loads a CSV, cleans it, and prints insights
1	Implement gradient descent + PCA from scratch in NumPy
2	End-to-end ML classifier: feature engineering → 3 models → evaluation
3	Build & train a CNN and an RNN from scratch in PyTorch
4	Tiny GPT from scratch → then a production RAG system with an agent
5	Deploy that RAG/agent as a monitored, evaluated, cost-aware API
6	Reproduce a paper or go deep on one situational area

Timeline & Reality Check

Tiers 0–5 are achievable in roughly **8–11 months** at 10–15 focused hours/week, and make you a strong, employable, application-focused AI engineer — effectively 100% of what that role needs. Tier 6 is a lifelong, just-in-time menu, not a checklist to finish. No syllabus is ever truly complete; the field changes monthly. The goal is enough structure to start, plus the judgment to learn the rest as real problems demand it.

*One rule above all: **build something at every stage**. The people who go furthest aren't those who studied the most theory — they're the ones who kept shipping projects while they learned.*